



HITEC University Taxila

Department of Cybersecurity

CS-317 Essentials of Information Security

Project Report

Submitted by:

Hassan Ali (23-cys-035)

Syed Haider Abbas (23-cys-038)

Submitted to:

Mr. Aashir Waqar

Contents

Secure Encrypted Chat System.....	4
Project Report	4
1. Introduction.....	4
2. Objectives	4
3. System Architecture	4
4. Technologies Used	5
4.1 Programming Language	5
4.2 Networking.....	5
4.3 Cryptography.....	5
4.4 GUI.....	5
4.5 Data Handling.....	5
5. Security Features	5
5.1 Encrypted Communication	5
5.2 Secure Authentication	5
5.3 File Encryption	5
5.4 Spam Detection	6
5.5 Phishing Detection	6
6. Functional Requirements.....	6
7. Non-Functional Requirements	6
8. Implementation Overview	6
8.1 Implementation and Screenshots.....	6
8.1.1 Server Execution Screenshot.....	6
8.1.2 Client Login Interface Screenshot	7
8.1.3 Successful Login and Chat Interface Screenshot.....	7
8.1.4 Encrypted Message Transmission Screenshot.....	8
8.1.5 Encrypted File Transfer Screenshot	8
8.1.6 Wireshark Encrypted Traffic Screenshot	8
8.2 Server Side.....	9
8.1 Server Side.....	9
8.2 Client Side	9
9. Testing and Results	9

9.1 Functional Testing	9
9.2 Security Testing.....	9
9.3 Error Handling	10
10. Limitations	10
11. Future Enhancements	10
12. Conclusion	10
13. References.....	10

Secure Encrypted Chat System

Project Report

Project Title: Secure Encrypted Chat System with File Transfer and Threat Detection

1. Introduction

In the modern digital era, secure communication is a critical requirement due to the increasing number of cyber threats such as eavesdropping, phishing, and data tampering. Traditional chat systems that transmit data in plain text are highly vulnerable to packet sniffing tools like Wireshark.

This project presents a **Secure Encrypted Chat System** that ensures confidentiality, integrity, and controlled access by using strong cryptographic techniques and secure socket communication. The system is designed as a client-server model with encrypted messaging, secure login, file transfer, and basic threat detection mechanisms.

2. Objectives

The main objectives of this project are:

- To design a secure client-server chat application
- To prevent plain-text data exposure over the network
- To implement strong encryption using AES-256
- To provide secure user authentication
- To support encrypted file sharing
- To detect spam and phishing attempts
- To ensure messages are unreadable in Wireshark

3. System Architecture

The system follows a **Client–Server Architecture**:

- **Server:**
 - Handles user authentication
 - Manages connected clients
 - Encrypts and decrypts all data
 - Broadcasts messages securely
- **Client:**
 - Provides a graphical user interface (GUI)
 - Encrypts messages before transmission
 - Decrypts received messages
 - Supports secure file transfer
 - All communication occurs over TCP sockets with encrypted payloads.

4. Technologies Used

4.1 Programming Language

- **Python 3** for rapid development and strong library support

4.2 Networking

- **Socket Programming (TCP)** for reliable communication

4.3 Cryptography

- **AES (Advanced Encryption Standard)**
 - Key Size: 256-bit
 - Mode: CFB (Cipher Feedback)
 - Ensures confidentiality of messages and files

4.4 GUI

- **Tkinter** for user-friendly client interface

4.5 Data Handling

- **Pickle Serialization** for structured message transfer
- **Length-Prefix Framing** to avoid TCP packet merging issues

5. Security Features

5.1 Encrypted Communication

- All messages are encrypted before sending
- Decryption occurs only at the intended recipient
- Wireshark captures show encrypted binary data only

5.2 Secure Authentication

- Username and password are validated on the server
- Unauthorized users are rejected immediately

5.3 File Encryption

- Files are encrypted using random AES keys
- Each file uses a unique key and IV
- Files are decrypted only after secure reception

5.4 Spam Detection

- Prevents rapid message flooding
- Blocks messages sent within a suspicious time interval

5.5 Phishing Detection

- Detects common phishing keywords
- Warns users when suspicious content is detected

6. Functional Requirements

- User login authentication
- Encrypted text messaging
- Encrypted file transfer
- Multiple client support
- Spam detection
- Phishing alert system

7. Non-Functional Requirements

- High confidentiality
- Reliable communication
- Easy-to-use interface
- Scalability for multiple users
- Resistance against packet sniffing

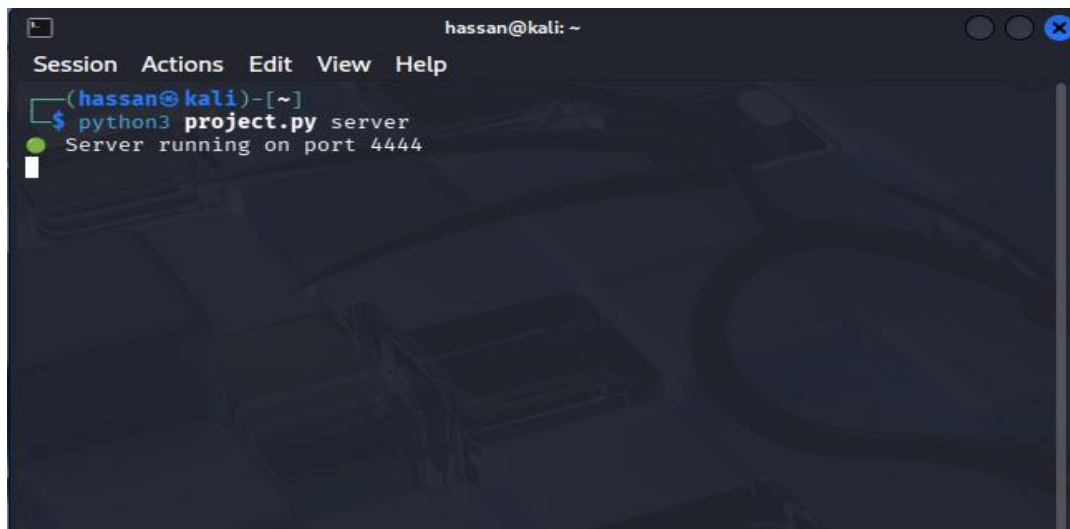
8. Implementation Overview

8.1 Implementation and Screenshots

(This section contains implementation screenshots of the Secure Encrypted Chat System. Screenshots include server startup, client login, successful authentication, encrypted message exchange, file transfer, and Wireshark packet analysis showing encrypted data.)

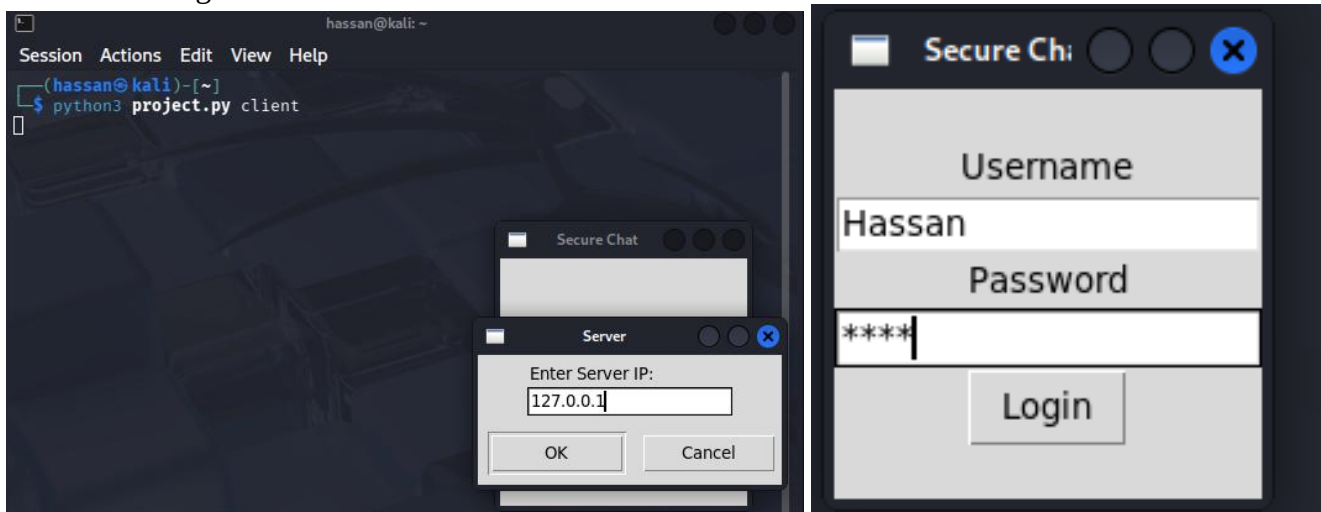
8.1.1 Server Execution Screenshot

- Server running and listening on the specified port



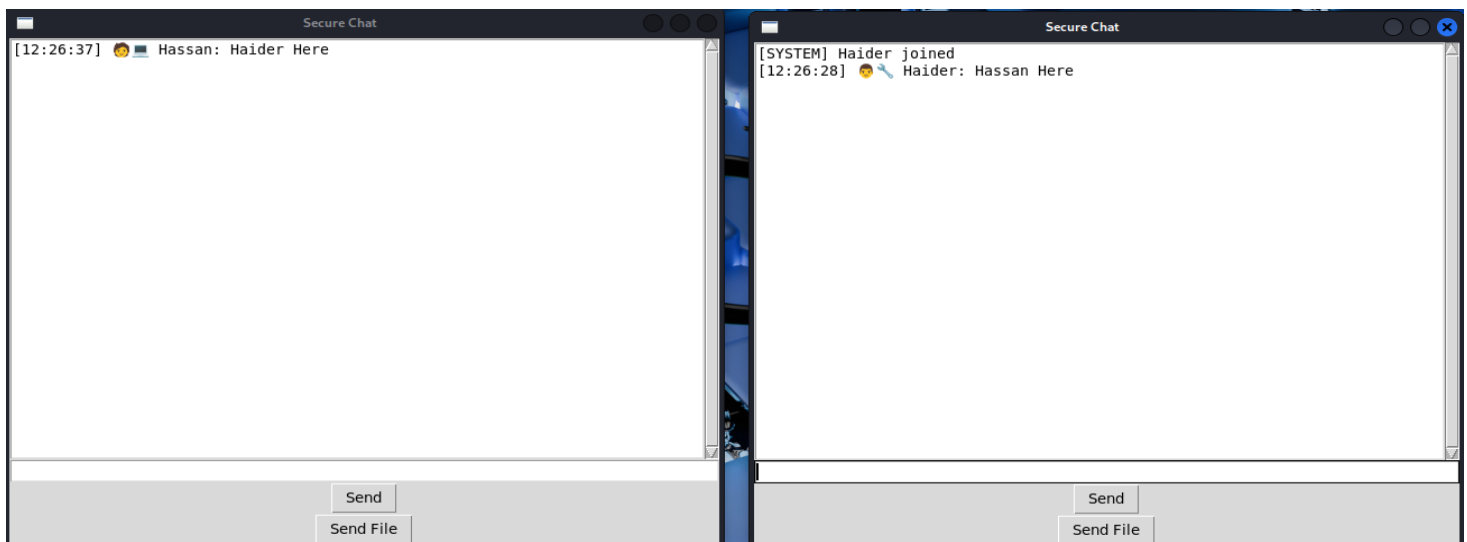
8.1.2 Client Login Interface Screenshot

- User login window



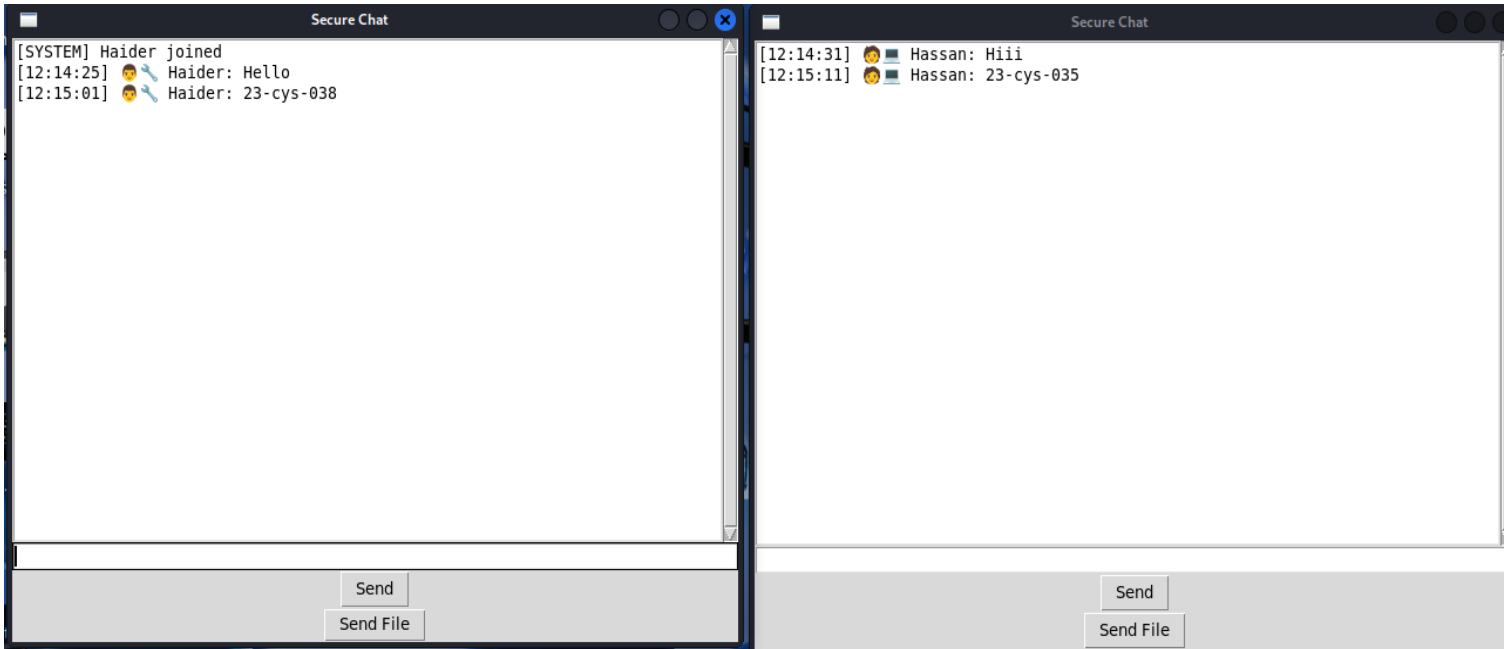
8.1.3 Successful Login and Chat Interface Screenshot

- Authenticated user inside chat window



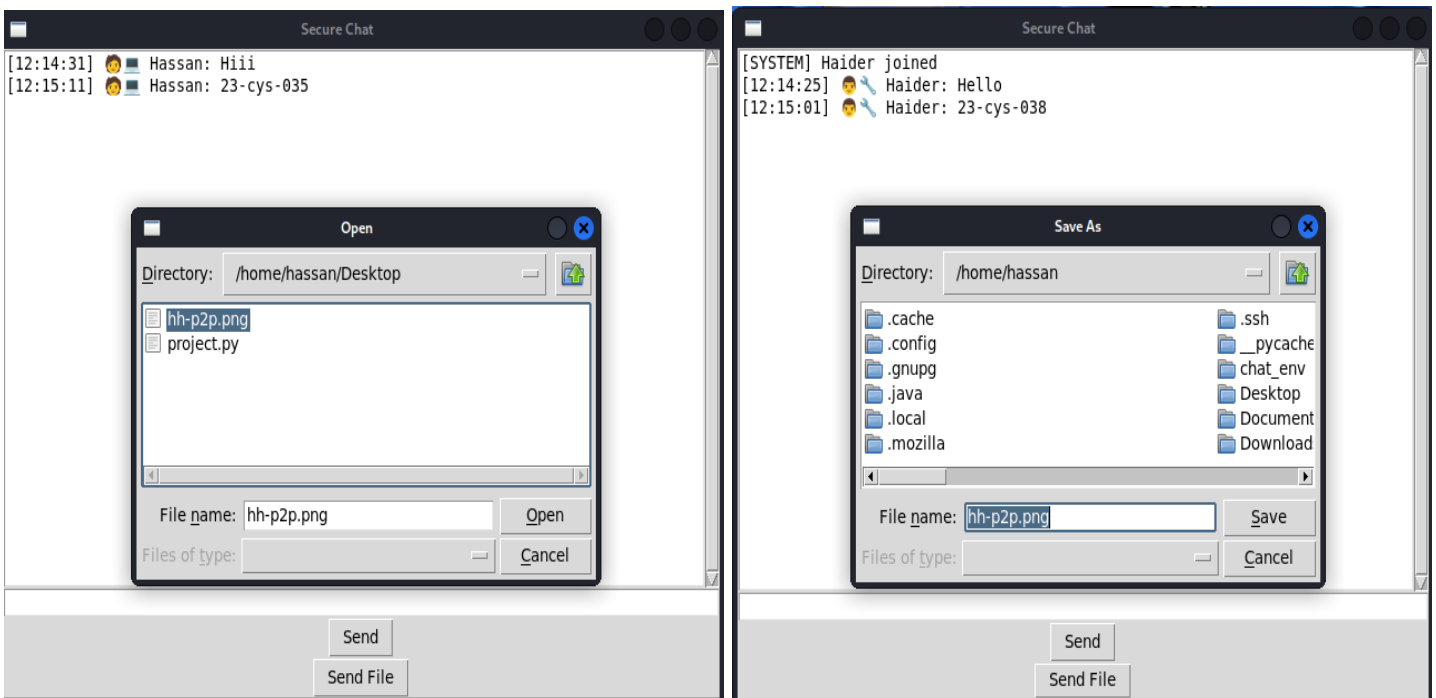
8.1.4 Encrypted Message Transmission Screenshot

- Chat messages exchanged between users



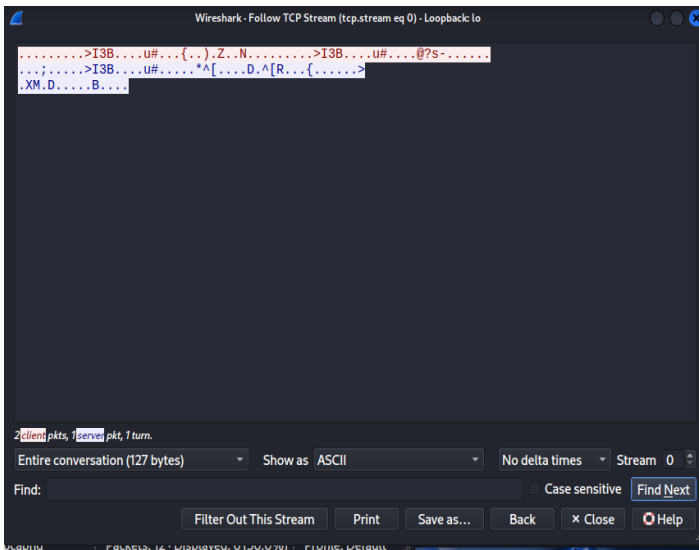
8.1.5 Encrypted File Transfer Screenshot

- File sent and received securely



8.1.6 Wireshark Encrypted Traffic Screenshot

- Captured packets showing unreadable encrypted payload
Text Message: File Transfer:



8.2 Server Side

8.1 Server Side

- Listens for incoming client connections
- Validates login credentials
- Manages active client sessions
- Encrypts and decrypts all data
- Broadcasts messages securely

8.2 Client Side

- Connects to server using IP and port
- Provides login interface
- Displays real-time chat messages
- Allows secure file sending and receiving

9. Testing and Results

9.1 Functional Testing

- Login success and failure scenarios tested
- Multiple clients tested simultaneously
- File transfer tested with different file types

9.2 Security Testing

- Wireshark packet analysis performed

- No plain-text credentials or messages visible
- All captured packets appeared encrypted

9.3 Error Handling

- Server unreachable conditions handled
- Invalid login attempts detected correctly

10. Limitations

- Passwords are stored in plain form (can be improved with hashing)
- Static session key (can be replaced with dynamic key exchange)
- No database integration

11. Future Enhancements

- Password hashing using bcrypt
- Diffie-Hellman key exchange
- TLS/SSL integration
- Database-backed user management
- Message history encryption and storage
- Role-based user permissions

12. Conclusion

This project successfully demonstrates a secure communication system using modern cryptographic techniques. By implementing AES-256 encryption and secure socket programming, the system effectively protects data from interception and unauthorized access.

The Secure Encrypted Chat System is a strong practical example of applying cybersecurity principles in real-world applications and provides a solid foundation for advanced secure communication systems.

13. References

- Python Documentation
- Cryptography Library Documentation
- Network Security Principles
- AES Encryption Standards